PROGRAMACIÓN AVANZADA

Hibernate

Parte III



Ing. Matias Macrino

HIBERNATE

Inclusión de entidades:

Cuando se trabaja con herramientas de JPA, nos puede resultar útil la inclusión de entidades dentro de otras entidades de tal forma que las dos ***compartan una única entidad.***

Anotaciones a utilizar:

@Embeddable : Con esta anotación, indicamos que la clase puede ser "integrable" dentro de una entidad.

@Embedded : Con esta anotación, indicamos que el campo o la propiedad de una entidad es una instancia de una clase que puede ser integrable. Es decir, para que funcione, el campo que hayamos anotado como ***@Embedded***, debe corresponderse con una clase que tenga la anotación ***@Embeddable.***

HIBERNATE

Inclusión de entidades - Ejemplo:

```java
@Embeddable
public class Direccion {

    private String calle;
    private String ciudad;
    private String codigoPostal;

    public Direccion() {

    public Direccion(String calle, String ciudad, String codigoPostal) {

    public String getCalle() {

    public void setCalle(String calle) {

    public String getCiudad() {

    public void setCiudad(String ciudad) {

    public String getCodigoPostal() {

    public void setCodigoPostal(String codigoPostal) {

}
```

Clase que será ***integrable*** a otras entidades.

HIBERNATE

Inclusión de entidades - Ejemplo:

Entidad que integra a la clase creada con anterioridad

```
@Entity
public class Persona implements Serializable{

    @Id
    private Integer id;
    private String nombre;
    @Embedded
    private Direccion direccion;

    public Persona() {

    public Persona(Integer id, String nombre, Direccion direccion) {

    public Integer getId() {

    public void setId(Integer id) {

    public String getNombre() {

    public void setNombre(String nombre) {

    public Direccion getDireccion() {

    public void setDireccion(Direccion direccion) {

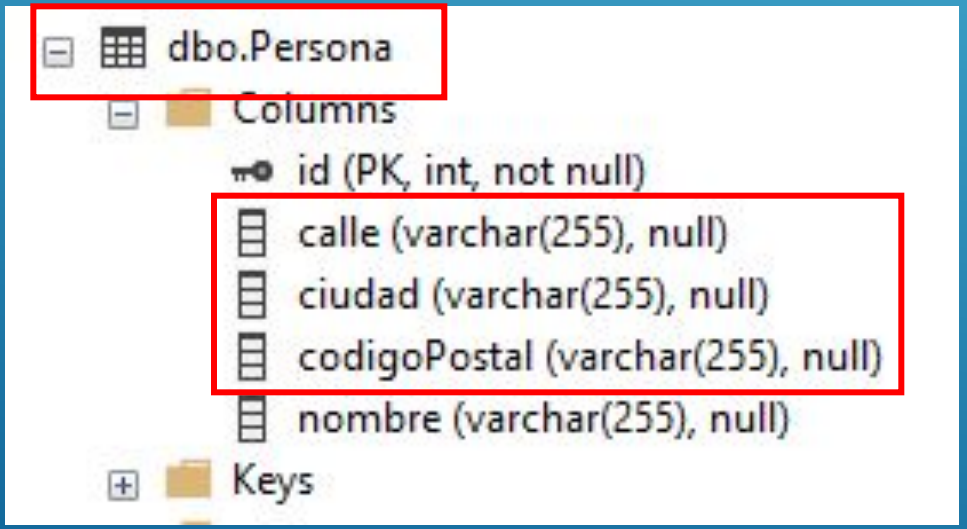
}
```

HIBERNATE

Inclusión de entidades - Ejemplo:

Realizamos la ejecución de ***Hibernate*** y al iniciar si no existe la ***entidad Persona*** en la base de datos, nos ejecuta el DDL de dicha tabla.

Lo que genera ***Hibernate/JPA*** usando ***la inclusión de entidades, es una única entidad "Persona", con los datos de clase "Direccion".***



HIBERNATE

Inclusión de entidades – Cambio de atributos:

Es posible cambiar el nombre de los campos de la clase a integrar. Lo cual se puede realizar a través de la siguiente Anotacion:

@AttributeOverrides : Con esta anotación, podemos sobreescribir o redefinir el mapeo de campos o propiedades de tipo básico. Esta anotación recibe un array de @AttributeOverride

@AttributeOverride: Con esta anotación, podemos redefinir el mapeo de tipos básicos. Su uso es @AttributeOverride(name="atributoDeLaClaseEmbebida", column = @Column(name="nombreColumnaEnLaTabla")),

HIBERNATE

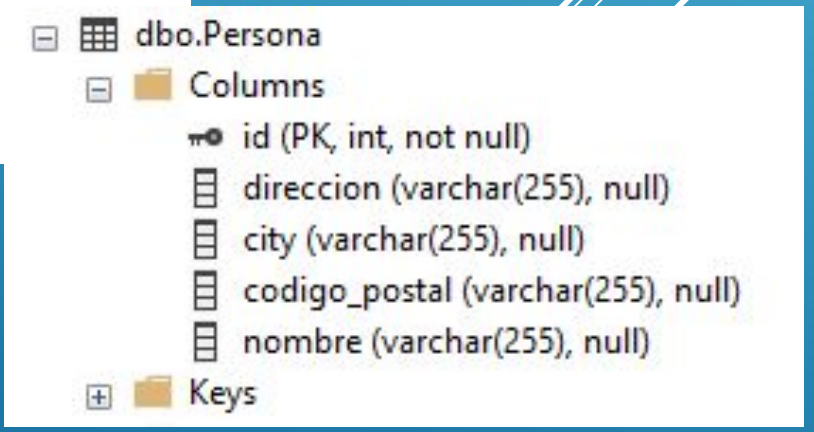
Inclusión de entidades – Cambio de atributos (Ejemplo):

```java
@Entity
public class Persona implements Serializable{

    @Id
    private Integer id;
    private String nombre;
    @Embedded
    @AttributeOverrides(value = {
            @AttributeOverride(name="calle", column=@Column(name="direccion")),
            @AttributeOverride(name="ciudad", column=@Column(name="city")),
            @AttributeOverride(name="codigoPostal", column=@Column(name="codigo_postal"))

        })
    private Direccion direccion;
```

Cambio de atributos



HIBERNATE

Query:

- ✔Una característica fundamental de cualquier ORM es la necesidad de leer dichos objetos de la base de datos.

- ✔***Hibernate*** tiene un objeto ***Query (org.hibernate.Query)*** que nos brinda el acceso a todas las funcionalidades para poder leer objetos desde la base de datos.

HIBERNATE

Query:

✔Lanzar una consulta con ***Hibernate*** es bastante simple:

- ✔ Usando el objeto ***Session*** llamamos al método "***createQuery(String queryString)***" con la consulta en formato ***HQL*** y cual nos retorna un objeto ***Query***.
- ✔ Sobre el objeto ***Query*** llamamos al método "***List()***" que nos retorna una lista de los objetos que ha retornado.

```
1   Query query = session.createQuery("SELECT p FROM Profesor p");
2   List<Profesor> profesores = query.list();
3   for (Profesor profesor : profesores) {
4       System.out.println(profesor.toString());
5   }
6
```

HIBERNATE

Query (list()):

- ***Lista de objetos:*** Como ya se menciono con anterioridad el método list() nos retorna una lista cono todos los objetos que son los resultados de la consulta realizada.

- En caso de no encontrar resultados se retornara una lista sin ningún elemento.

HIBERNATE

Query (list()):

- ***Lista Array de objetos:*** Tambien se permite que las consultas retornen datos escalares en vez de clases completas.
  - Ejemplo: "SELECT p.id,p.nombre FROM Profesor p"

- En el ejemplo se visualiza como en vez de retornar un objeto Profesor se retorna únicamente el código de profesor y su nombre.

- En estos casos el método list() retorna una lista con un Array de objetos con tantos elementos como propiedades hayamos indicado en el SELECT.

HIBERNATE

Query (list()):

✔*Lista Array de objetos:*

```
1   Query query = session.createQuery("SELECT p.id,p.nombre FROM Profesor p");
2   List<Object[]> listDatos = query.list();
3   for (Object[] datos : listDatos) {
4       System.out.println(datos[0] + "--" + datos[1]);
5   }
6
```

En la línea 4 podemos ver como se accede a los dos datos de cada fila mediante el índice de cada propiedad establecida en el SELECT (datos[0] y datos[1])

HIBERNATE

Query (uniqueResult()):

- ✔Nos encontraremos en muchas ocasiones en que una consulta únicamente retornara cero o un resultado. En ese caso es poco practico que nos retorne una lista con un único elemento.

- ✔Para estos casos ***Hibernate*** dispone del método ***uniqueResult().***

- ✔En caso de no encontrar resultados retorna ***null.***

HIBERNATE

Query (uniqueResult()):

```
1  Profesor profesor = (Profesor) session.createQuery("SELECT p FROM Profesor p WHERE id=1001").uniqueResult();
2  System.out.println("Profesor con Id 1001=" + profesor.getNombre());
3
```

✔Siempre se debe comprobar si ha retornado o no ***null***.

✔Si dicho método retornara mas de un resultado se producirá la Excepcion ***"org.hibernate.NonUniqueResultException"***

HIBERNATE

HQL (Hibernate Query Languaje):

- ✔Es el lenguaje de consultas que usa Hibernate para lo obtención de objetos desde la BD.

- ✔Su principal característica es que las consultas se realizan sobre los objetos Java que forman nuestro modelo de negocio.

HIBERNATE

HQL (Hibernate Query Languaje):

✔Características:

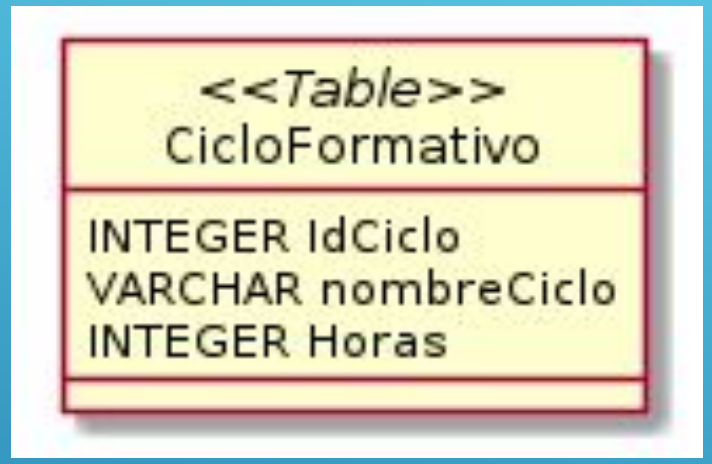
- ✔ Los tipos de datos son los de Java.
- ✔ Las consultas son independientes del lenguaje de SQL especifico de la BD.
- ✔ Las consultas son independientes del modelo de tablas de la base de datos.
- ✔ Es posible tratar con las colecciones Java.
- ✔ Es posible navegar entre los distintos objetos en la propia consulta.

HIBERNATE

HQL (Hibernate Query Languaje):



Obj. Negocio



Base de datos

- ✔ Para la clase Ciclo su tabla se llama CicloFormativo
- ✔ En la clase Ciclo su propiedad nombre se almacena en la columna llamada nombreCiclo
- ✔ Consulta HQL:

SELECT c FROM Ciclo c ORDER BY nombre

HIBERNATE

HQL (Hibernate Query Languaje):

*Diferencias entre **HQL** y **SQL**:*

- Ciclo hace referencia a la clase Ciclo y NO a la tabla CicloFormativo. Notese que la clase Java y la tabla tienen distinto nombre .
- Es necesario definir el alias c de la clase Java Ciclo.
- Tras la palabra ***SELECT*** se usa el alias en vez del "*".
- Al ordenar los objetos se usa la propiedad nombre de la clase Ciclo en vez de la columna nombreCiclo de la tabla CicloFormativo.

HIBERNATE

HQL (Hibernate Query Languaje):

- ✔Si o si hay que incluir el alias en la consulta HQL. Si no se hace y se deja la consulta de la siguiente manera: ***SELECT Ciclo FROM Ciclo,*** se producirá la siguiente Excepción: ***java.lang.NullPointerException***

- ✔***Hibernate*** soporta no incluir la sentencia SELECT de SQL, en la consulta HQL, quedando la consulta de la siguiente manera: ***FROM Ciclo.***

HIBERNATE

HQL (Case Sensitive):

✔El lenguaje HQL es por demasiado Case Sensitive. Por lo cual hay que prestar mucha atención a la sintaxis utilizada. Ejemplos:

```
1   select count(*) from Ciclo
2   SELECT COUNT(*) FROM Ciclo
3
```

Consultas equivalentes. Palabras claves no son Case Sensitive

```
1   SELECT c.Nombre FROM Ciclo c WHERE Nombre='Desarrollo de aplicaciones Web'
2   SELECT c.nombre FROM ciclo c WHERE nombre='Desarrollo de aplicaciones Web'
3
```

Consultas erróneas. Los nombres de la clase y sus propiedades sin son Case Sensitive.
En la línea 1 la propiedad nombre no es con "N".
En la línea 2 la clase Java es Ciclo con "C" y no con "c"

```
1   SELECT c.nombre FROM Ciclo c WHERE nombre='Desarrollo de aplicaciones Web'
2   SELECT c.nombre FROM Ciclo c WHERE nombre='DESARROLLO DE APLICACIONES WEB'
3
```

Consultas equivalentes. Criterios de búsqueda no son Case Sensitive

HIBERNATE

HQL (Filtros – Clausula WHERE):

- ✔***String***: Para indicar que es un literal de texto se escribe entre comillas simples al igual que SQL. Ejemplo: ***WHERE nombre = 'Texto'***
- ✔***Integer:*** Para indicar que es un literal del tipo entero se hace directamente con el numero. Ejemplo ***WHERE id = 3***
- ✔***Long:*** Para indiciar que es un numero del tipo Long se hace con el numero mas la letra L en mayúscula. Ejemplo ***WHERE id = 4L***
- ✔***Double:*** Para indicar que es un double se hace separando la parte con un punto. Ejemplo ***WHERE valor = 1.45***
- ✔***Float:*** Para indicar que es un float se hace separando la parte decimal con un punto y agregándole la letra F en mayúscula al final. Ejemplo ***WHERE valor = 1.45F***
- ✔***Fecha:*** Para indicar una fecha se hace entre comillas simples con el formato "yyyy-mm-dd". Ejemplo: ***WHERE fecha = '2017-11-01'***

HIBERNATE

HQL:

✔***Dicho lenguaje también nos permite:***

- ***Ordenar los datos:*** Como en SQL también es posible ordenar los resultados de la consulta usando la palabra ***ORDER BY.***

- ***Agrupar los datos:*** Como en SQL se puede realizar agrupaciones mediante las palabras claves ***GROUP BY y HAVING***

- ***Subconsultas: HQL también soporta subconsultas tal como se puede hacer con SQL.***

HIBERNATE

HQL (Parámetros):

✔***Posicional:*** Esta forma de definir los parámetros es idéntica a la utilizada con la interfaz ***PreparedStatement de JDBC.***

```
1   String nombre="Jose";
2   String ape1="Lopez";
3   String ape2="Garcia";
4
5   Query query = session.createQuery("SELECT p FROM Profesor p where nombre=? AND ape1=? AND ape2=?");
6   query.setString(0,nombre);
7   query.setString(1,ape1);
8   query.setString(2,ape2);
9
```

✔En caso de la propiedad no ser String se utiliza el tipo correspondiente. Ejemplo: ***setInt(índice, valor)***

HIBERNATE

HQL (Parámetros):

✔***Nombres:*** En este caso los parámetros se definen como nombre precedidos de dos puntos ":". Esto hace que el código sea más legible y menos propenso a error.

```
1  String nombre="Jose";
2  String ape1="Lopez";
3  String ape2="Garcia";
4
5  Query query = session.createQuery("SELECT p FROM Profesor p where nombre=:nombre AND ape1=:ape1 AND ape2=:ape2");
6  query.setString("nombre",nombre);
7  query.setString("ape1",ape1);
8  query.setString("ape2",ape2);
9
```

✔En caso de la propiedad no ser String se utiliza el tipo correspondiente. Ejemplo: ***setInt(nombre, valor)***

HIBERNATE

HQL (Parámetros):

✔***Mejoras:*** Existe otra forma de asignar valores a los parámetros que es independiente del tipo. Los métodos son ***setParameter(int position,Object val)*** y ***setParameter(String name,Object val)*** según si el parámetro es por índice o por nombre.

```
1  String nombre="Jose";
2  String ape1="Lopez";
3  String ape2="Garcia";
4
5  Query query = session.createQuery("SELECT p FROM Profesor p where nombre=:nombre AND ape1=:ape1 AND ape2=:ape2");
6  query.setParameter("nombre",nombre);
7  query.setParameter("ape1",ape1);
8  query.setParameter("ape2",ape2);
9
```

HIBERNATE

HQL (Parámetros):

✔*Mejoras encadenado:*

```
1  String nombre="Jose";
2  String ape1="Lopez";
3  String ape2="Garcia";
4
5  Query query = session.createQuery("SELECT p FROM Profesor p where nombre=:nombre AND ape1=:ape1 AND ape2=:ape2")
6      .setParameter("nombre",nombre).setParameter("ape1",ape1).setParameter("ape2",ape2);
7
```

```
1  String nombre="Jose";
2  String ape1="Lopez";
3  String ape2="Garcia";
4
5  Query query = session.createQuery("SELECT p FROM Profesor p where nombre=:nombre AND ape1=:ape1 AND ape2=:ape2")
6      .setParameter("nombre",nombre)
7      .setParameter("ape1",ape1)
8      .setParameter("ape2",ape2);
9
```